

Apache Iceberg

Hive-Metastore Module

Design Document

Architecture / Design Patterns / Call Chain Analysis

Version: 1.9.2

Based on apache-iceberg-1.9.2 branch

Module: iceberg-hive-metastore

Package: org.apache.iceberg.hive

17 Core Source Files | 16 Test Files

Contents

1. Module Overview
2. Architecture Design
 - 2.1 Module Hierarchy
 - 2.2 Core Architecture Diagram
 - 2.3 Class Inheritance Hierarchy
3. Core Process Flows
 - 3.1 Catalog Initialization
 - 3.2 Table Commit (doCommit)
 - 3.3 Distributed Lock Acquisition
 - 3.4 Table Metadata Refresh (doRefresh)
4. Sequence Diagrams
 - 4.1 Create Table Sequence
 - 4.2 Commit with Lock Sequence
 - 4.3 Concurrent Commit Handling
5. Design Patterns
 - 5.1 Strategy Pattern - Lock Mechanism
 - 5.2 Template Method Pattern
 - 5.3 Object Pool Pattern
 - 5.4 Adapter Pattern
 - 5.5 Decorator Pattern
 - 5.6 Builder Pattern
6. Core Class Analysis
 - 6.1 HiveCatalog
 - 6.2 HiveTableOperations
 - 6.3 MetastoreLock
 - 6.4 CachedClientPool
 - 6.5 HMSTablePropertyHelper
 - 6.6 HiveOperationsBase
7. Key Scenario Call Chains
 - 7.1 CREATE TABLE
 - 7.2 INSERT / APPEND DATA
 - 7.3 ALTER TABLE (Schema Evolution)
 - 7.4 DROP TABLE
 - 7.5 Concurrent Write Conflict
8. Configuration Reference

9. Summary

1. Module Overview

Apache Iceberg hive-metastore module is the bridge connecting the Iceberg table format with the Hive Metastore (HMS) service. This module implements a complete Catalog interface that enables managing Iceberg tables, views, and namespaces through the Hive Metastore as the metadata catalog backend. It is one of the most commonly used Catalog implementations in production environments.

The core responsibility of this module includes: (1) Storing Iceberg metadata file locations in HMS table properties, using HMS as the single source of truth for metadata pointers; (2) Providing distributed lock mechanisms to ensure concurrent commit safety in multi-writer scenarios; (3) Bi-directional schema conversion between Iceberg types and Hive types; (4) Connection pool management for efficient HMS client lifecycle handling.

1.1 Source File Overview

File	Lines	Role
HiveCatalog.java	939	Entry point: Catalog interface impl, table/view/namespace CRUD
HiveTableOperations.java	616	Table metadata commit/refresh, encryption, lock integration
MetastoreLock.java	541	Distributed lock: JVM lock + HMS lock + heartbeat
HiveViewOperations.java	368	View metadata commit/refresh
HMSTablePropertyHelper.java	323	Iceberg metadata to HMS property synchronization
HiveOperationsBase.java	235	Common interface for table/view operations
CachedClientPool.java	218	Cached HMS client pool with UGI/config-based keys
HiveSchemaUtil.java	192	Bidirectional Iceberg-Hive schema conversion
HiveSchemaConverter.java	159	Hive TypeInfo to Iceberg Type conversion impl
HiveClientPool.java	118	IMetaStoreClient pool: create, reconnect, close
MetastoreUtil.java	89	Cross-version alterTable via reflection
HiveVersion.java	68	Hive version detection enum
HiveHadoopUtil.java	48	Hadoop user info utility
NoLock.java	45	No-op lock impl (HIVE-26882)
HiveLock.java	28	Lock interface: lock/ensureActive/unlock
LockException.java	34	Lock operation runtime exception
RuntimeMetaException.java	40	MetaException wrapper

2. Architecture Design

2.1 Module Hierarchy

The hive-metastore module follows a clear layered architecture with well-defined responsibilities at each level. The Catalog Layer serves as the entry point, the Operations Layer handles metadata read/write against HMS, the Infrastructure Layer provides connection pooling, locking, and schema conversion, and the Utility Layer contains cross-cutting helpers.

```

| APPLICATION / ENGINE LAYER |
| (Spark / Flink / Trino / Hive Engine) |
+=====+
| |
| Catalog SPI (catalog.Catalog) |
| |
+=====+
| 1. CATALOG LAYER (Entry Point) |
| |
| +-----+ builds +-----+ |
| | HiveCatalog |----->| ViewAwareTableBuilder | |
| | (BaseMetastoreView | | TableAwareViewBuilder | |
| | Catalog) | +-----+ |
| +-----+ |
| | newTableOps() | newViewOps() |
+=====+
| |
+=====+
| 2. OPERATIONS LAYER (Metadata R/W) |
| |
| +-----+ +-----+ |
| | HiveTableOperations | | HiveViewOperations | |
| | (BaseMetastoreTable | | (BaseViewOperations | |
| | Operations) | | + HiveOpsBase) | |
| +-----+ +-----+ |
| | implements HiveOperationsBase | |
| +-----+ +-----+ |
+=====+
| |
+=====+
| 3. INFRASTRUCTURE LAYER (Cross-Cutting) |
| |
| +-----+ +-----+ +-----+ |
| | MetastoreLock | | CachedClient | | HMSTablePropertyHelper | |
| | NoLock | | Pool | | (metadata -> HMS props) | |
| | (HiveLock) | | HiveClientPool | +-----+ |
| +-----+ +-----+ |
| |
| +-----+ +-----+ +-----+ |
| | HiveSchemaUtil | | MetastoreUtil | | HiveVersion | |
| | HiveSchemaConv. | | (alter_table) | | (detection) | |
| +-----+ +-----+ +-----+ |
+=====+
| |
| Hive Metastore (Thrift RPC) |

```

2.2 Core Architecture Diagram

The core architecture revolves around a metadata-pointer pattern: Iceberg stores its actual table metadata (schema, partitions, snapshots) in JSON files on distributed storage (HDFS/S3), while HMS only stores the pointer (metadata_location) to the latest metadata file. This design means HMS acts purely as a coordination service - it does not need to understand Iceberg's internal format. Each commit atomically updates this pointer in HMS, protected by a distributed lock.

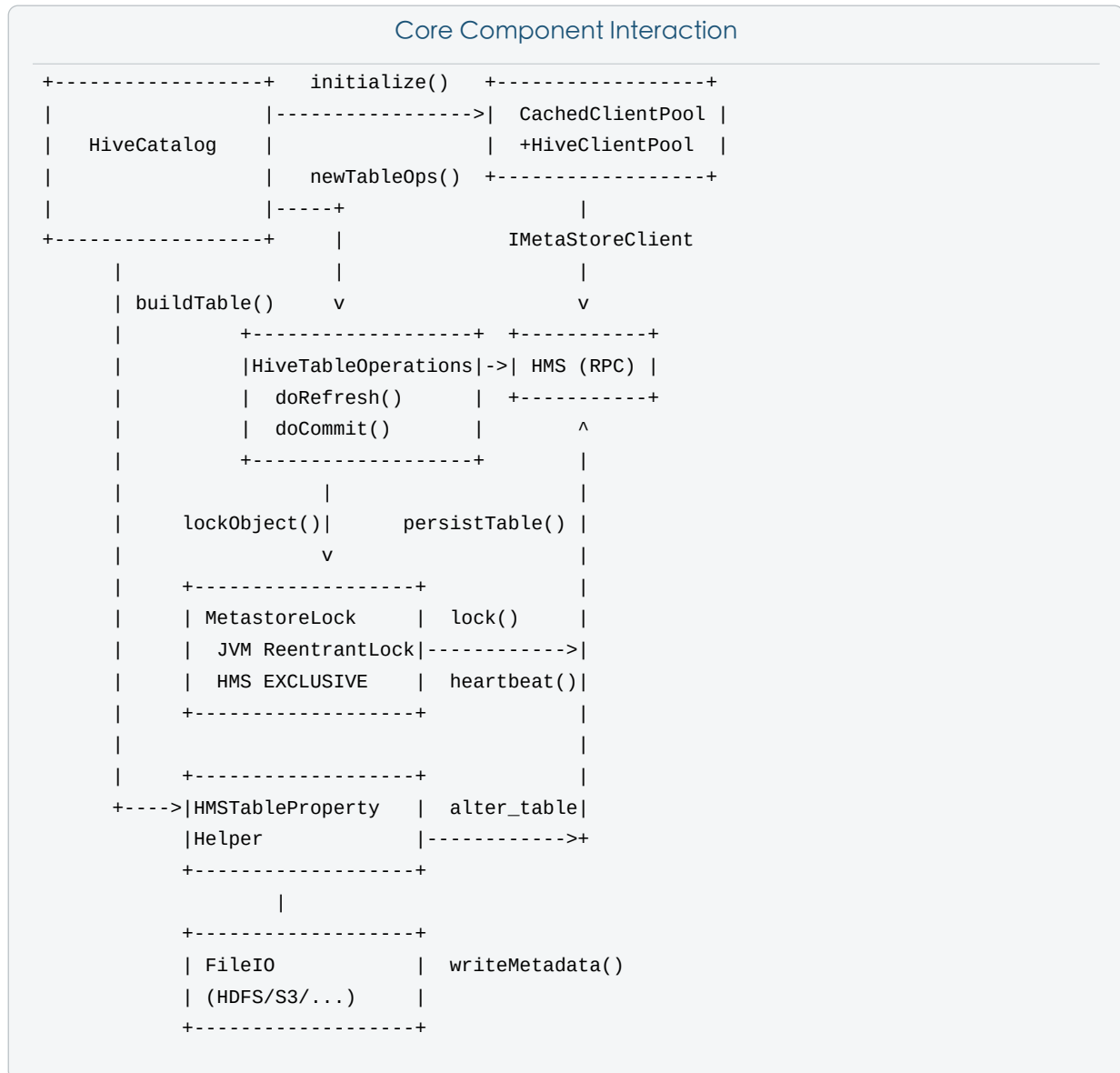


Figure 2.2: Core Component Interaction Diagram

2.3 Class Inheritance Hierarchy

Class Inheritance Hierarchy

```
Catalog (API interface)
|-- BaseMetastoreCatalog
    |-- BaseMetastoreViewCatalog
        |-- HiveCatalog (+SupportsNamespaces, +Configurable)

TableOperations (API interface)
|-- BaseMetastoreTableOperations
    |-- HiveTableOperations (+HiveOperationsBase)

ViewOperations (API interface)
|-- BaseViewOperations
    |-- HiveViewOperations (+HiveOperationsBase)

ClientPool<IMetaStoreClient, TException> (API interface)
|-- ClientPoolImpl<IMetaStoreClient, TException>
    |-- HiveClientPool (actual pool with create/reconnect/close)
    |-- CachedClientPool (cache layer, delegates to HiveClientPool)

HiveLock (package-private interface)
|-- MetastoreLock (JVM lock + HMS distributed lock + heartbeat)
|-- NoLock (no-op, for Hive >= 2 with HIVE-26882)

HiveOperationsBase (interface with default methods)
|-- HiveTableOperations (implements)
|-- HiveViewOperations (implements)
```

Figure 2.3: Complete Class Inheritance Tree

3. Core Process Flows

3.1 Catalog Initialization Flow

When an application starts using HiveCatalog, the initialization process sets up all required infrastructure: HMS connection, file I/O, encryption, and client pooling.



Figure 3.1: HiveCatalog Initialization Flow

3.2 Table Commit Flow (doCommit)

The doCommit method in HiveTableOperations is the most critical path in the entire module. It implements an optimistic concurrency control mechanism with distributed locking. The commit process follows a strict sequence: write metadata to storage first, then acquire a lock, verify no concurrent modifications, update HMS atomically, and finally release the lock. This ensures that if the process crashes after writing metadata but before updating HMS, the orphaned metadata file is cleaned up.

Table Commit Flow

```
HiveTableOperations.doCommit(base, metadata)
|
+-- 1. Handle encryption (if StandardEncryptionManager)
|   +-- Add encryption keys to metadata via builder
|
+-- 2. Write new metadata file to storage
|   +-- writeNewMetadataIfRequired(newTable, metadata) -> newMetadataLocation
|
+-- 3. Determine lock strategy
|   +-- lockObject(metadata) -> MetastoreLock or NoLock
|
+-- 4. ACQUIRE LOCK (lock.lock())
|   +-- JVM ReentrantLock (process-level)
|   +-- HMS EXCLUSIVE lock (cluster-level)
|   +-- Start heartbeat thread
|
+-- 5. Load current HMS table
|   +-- If exists + newTable: throw AlreadyExistsException
|   +-- If not exists: create new HMS Table object
|
+-- 6. Verify no concurrent modification
|   +-- Compare base.metadataFileLocation() with HMS metadata_location
|   +-- Mismatch -> throw CommitFailedException
|
+-- 7. Update HMS table properties
|   +-- HMSTablePropertyHelper.updateHmsTableForIcebergTable()
|   +-- Set schema, partition spec, sort order, snapshot stats
|
+-- 8. lock.ensureActive() -- verify heartbeat still OK
|
+-- 9. Persist to HMS
|   +-- persistTable(tbl, updateHiveTable, envContext)
|   +-- createTable() or alterTable() via Thrift
|
+-- 10. lock.ensureActive() -- final check
|
+-- 11. [FINALLY] Cleanup + Unlock
|   +-- On FAILURE: delete orphaned metadata file
|   +-- lock.unlock() (HMS unlock + JVM unlock)
```

Figure 3.2: doCommit Complete Flow

3.3 Distributed Lock Acquisition Flow

MetastoreLock implements a two-tier locking mechanism. The first tier is a JVM-level ReentrantLock cached per table name, preventing concurrent commits from the same process. The second tier is an HMS-level EXCLUSIVE lock using the Hive Metastore's built-in lock API. A heartbeat thread keeps the HMS lock alive during long-running operations. If the lock is in WAITING state, the system retries with exponential backoff until timeout.

Lock Acquisition and Release Flow

```
MetastoreLock.lock()
|
+-- 1. acquireJvmLock()
|   +-- commitLockCache.get(fullName, -> new ReentrantLock(true))
|   +-- jvmLock.lock() // blocking, fair ordering
|
+-- 2. acquireLock() -> hmsLockId
|   |
|   +-- 2a. createLock()
|       |   +-- Build LockRequest(EXCLUSIVE, TABLE level)
|       |   +-- Set agentInfo = 'Iceberg-' + UUID
|       |   +-- metaClients.run(client -> client.lock(request))
|       |   +-- On TException: findLock() by agentInfo (Hive 2+)
|       |   +-- Retry with exponential backoff (50ms~5s, 3min timeout)
|       |
|       +-- 2b. If lockState == WAITING:
|           +-- Poll: metaClients.run(client.checkLock(lockId))
|           +-- Retry: exponential backoff (50ms~5s, 3min timeout)
|           +-- On timeout: unlock + throw LockException
|
+-- 3. Start Heartbeat
    +-- new Heartbeat(metaClients, lockId, 4min interval)
    +-- schedule at fixedRate: intervalMs/2 initial, intervalMs period
    +-- Heartbeat.run(): client.heartbeat(0, lockId)

MetastoreLock.unlock()
|
+-- 1. heartbeat.cancel() + scheduler.shutdown()
+-- 2. metaClients.run(client.unlock(lockId))
+-- 3. releaseJvmLock(): jvmLock.unlock()
```

Figure 3.3: MetastoreLock Lifecycle

3.4 Table Metadata Refresh Flow (doRefresh)

Table Metadata Refresh Flow

```
HiveTableOperations.doRefresh()
|
+-- 1. metaClients.run(client.getTable(db, table))
|   +-- On NoSuchObjectException + no current: return (table not created)
|   +-- On NoSuchObjectException + has current: throw NoSuchTableException
|
+-- 2. Validate table type
|   +-- validateIcebergViewNotLoadedAsIcebergTable(table)
|   +-- validateTableIsIceberg(table) // check table_type=ICEBERG
|
+-- 3. Extract properties from HMS
|   +-- metadataLocation = params.get('metadata_location')
|   +-- tableKeyId = params.get('encryption.table-key')
|   +-- dekLength = params.get('encryption.dek-length')
|   +-- metadataHash = params.get('metadata_hash')
|
+-- 4. refreshFromMetadataLocation(metadataLocation, maxRetries=2)
|   +-- Read metadata JSON from storage via FileIO
|   +-- Parse into TableMetadata object
|
+-- 5. Handle encryption (if tableKeyId != null)
|   +-- checkIntegrityForEncryption(keyId, dekLen, hash)
|   +-- Merge encryption keys from metadata + encryptionManager
|   +-- Force re-creation of encryptionManager
```

Figure 3.4: doRefresh Flow

4. Sequence Diagrams

4.1 Create Table Sequence

When a user creates an Iceberg table through HiveCatalog, the following sequence of interactions occurs between the major components. The Catalog first checks for view conflicts, then delegates to the commit mechanism which writes metadata and registers the table in HMS atomically.

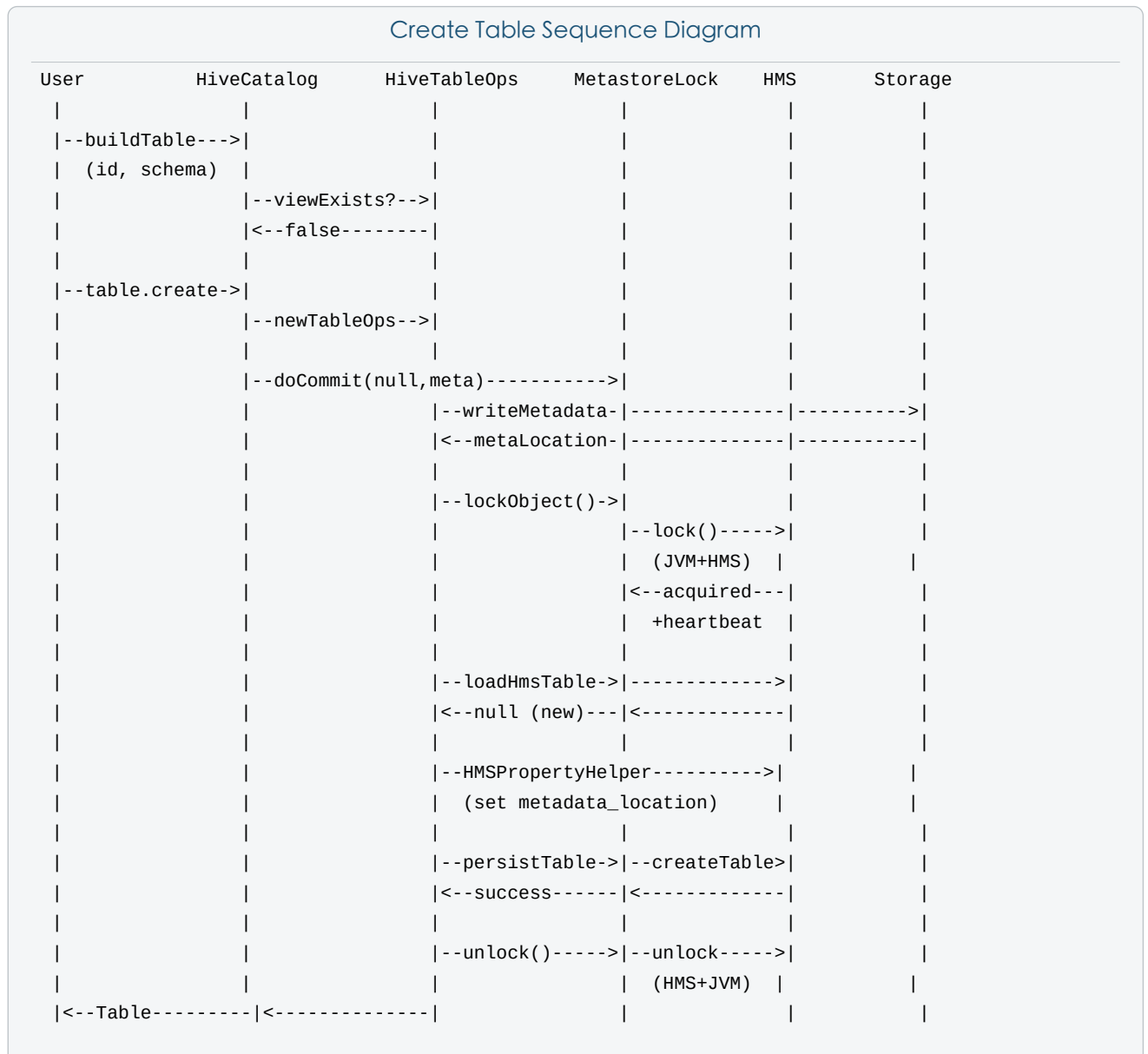


Figure 4.1: Create Table Interaction Sequence

4.2 Commit with Lock Sequence

During a normal data commit (e.g., INSERT INTO), the process writes new data files and a new metadata file, then acquires a distributed lock to atomically update the metadata pointer in HMS. The heartbeat ensures the lock remains active during the commit window.

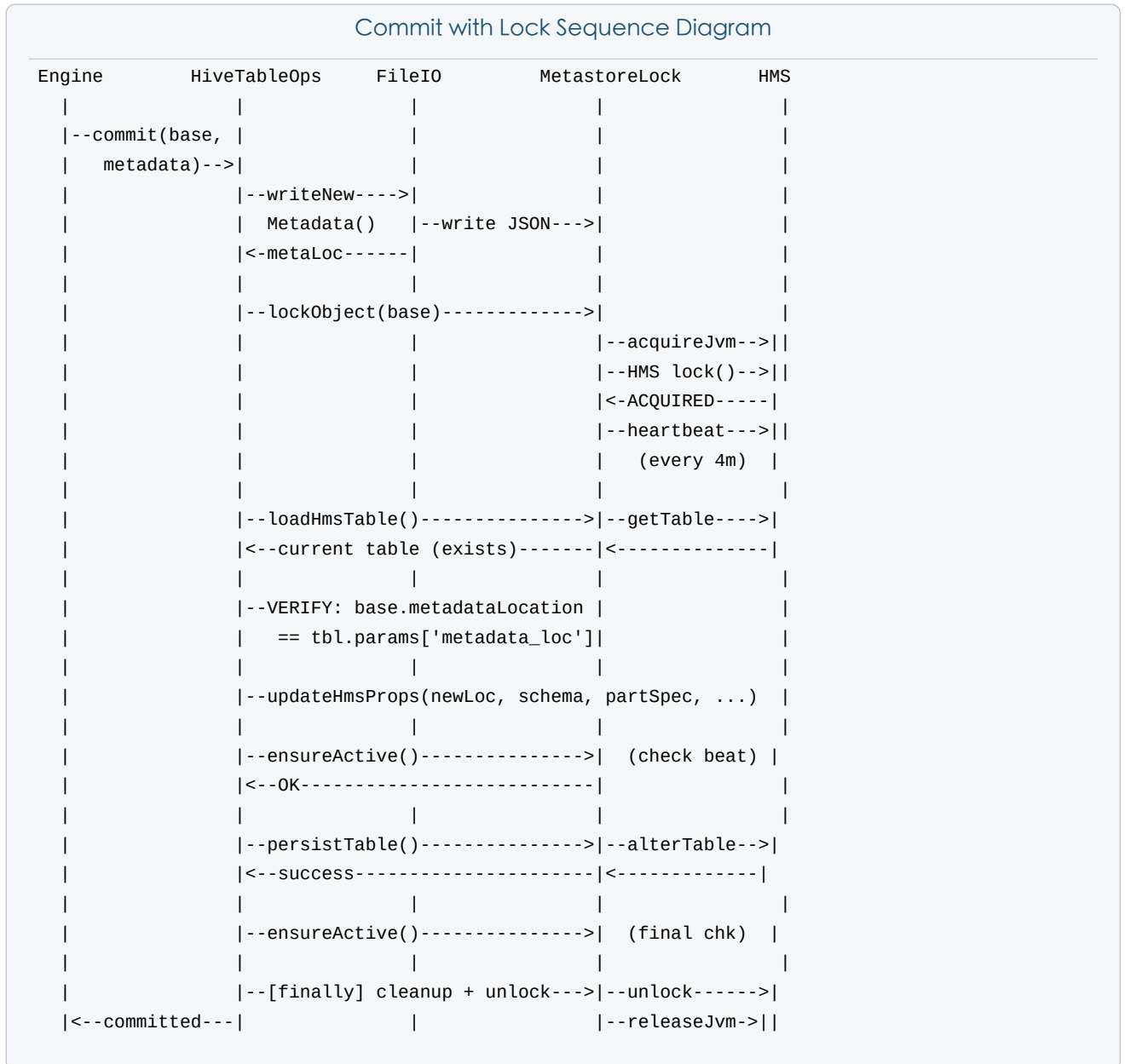


Figure 4.2: Data Commit with Lock Sequence

4.3 Concurrent Commit Handling

When two processes attempt to commit to the same table concurrently, the distributed lock serializes the operations. The second writer detects the conflict via metadata location mismatch and receives a `CommitFailedException`. For unknown commit states (e.g., network errors), the system performs a `checkCommitStatus` to determine the actual outcome.

Concurrent Commit Conflict Handling

Writer-A	Writer-B	MetastoreLock	HMS
--writeMetadata-->			
(v1 -> v2)			
	--writeMetadata-->		
	(v1 -> v3)		
--lock()----->			
	(JVM+HMS)	--lock ACQUIRED->	
	--lock()----->		
	BLOCKED (JVM)		
--verify(v1==v1)->			
OK			
--alterTable(v2)->			
SUCCESS			
--unlock()----->			
	--lock acquired-->		
	--verify(v1==v2)->		
	MISMATCH!		
	--CommitFailedException		
	'Base metadata v1 != current v2'		
	--cleanup v3 meta		
	--unlock()----->		

Figure 4.3: Concurrent Commit Conflict Detection

5. Design Patterns

5.1 Strategy Pattern - Lock Mechanism

The lock mechanism uses the Strategy Pattern, allowing the commit process to work with different lock implementations without code changes. The HiveLock interface defines the contract (lock/ensureActive/unlock), and concrete strategies are selected at runtime based on configuration.

Strategy Pattern: Lock Selection

```
+-----+
| <<interface>> HiveLock |
+-----+
| + lock(): void          |
| + ensureActive(): void  |
| + unlock(): void        |
+-----+
      ^           ^
      |           |
+-----+ +-----+
| MetastoreLock | | NoLock          |
+-----+ +-----+
| JVM lock      | | lock()    -> nop |
| HMS lock      | | ensure()  -> nop |
| heartbeat     | | unlock()  -> nop |
+-----+ +-----+
```

```
Selection logic in HiveTableOperations.lockObject():
  if hiveLockEnabled(metadata, conf) -> new MetastoreLock(...)
  else                               -> new NoLock()
```

This pattern enables zero-disruption migration: environments with Hive ≥ 2 (HIVE-26882) can use NoLock to rely on HMS's own CAS-based concurrency control, while older environments use MetastoreLock for distributed coordination. The selection is controlled by the iceberg.hive.lock-enabled table property or ConfigProperties.LOCK_HIVE_ENABLED.

5.2 Template Method Pattern

Both HiveTableOperations and HiveViewOperations follow the Template Method Pattern inherited from their respective base classes. The base class defines the overall commit/refresh algorithm, while subclasses implement specific steps (doCommit, doRefresh). BaseMetastoreTableOperations provides common utilities like writeNewMetadataIfRequired, refreshFromMetadataLocation, and checkCommitStatus.

Template Method Pattern

```
+-----+
| BaseMetastoreTableOperations |
+-----+
| + commit(base, metadata) [final] | <- Template Method
|   |-- doCommit(base, metadata) | <- Abstract hook
| + refresh() [final] | <- Template Method
|   |-- doRefresh() | <- Abstract hook
|
| # writeNewMetadataIfRequired() | <- Shared utility
| # refreshFromMetadataLocation() | <- Shared utility
| # checkCommitStatus() | <- Shared utility
+-----+
      ^
      |
+-----+
| HiveTableOperations |
+-----+
| # doCommit(base, metadata) | <- HMS-specific impl
|   |-- acquire lock |
|   |-- verify metadata location |
|   |-- persist to HMS |
| # doRefresh() | <- HMS-specific impl
|   |-- getTable from HMS |
|   |-- validate Iceberg type |
+-----+
```

5.3 Object Pool Pattern - Connection Pool

The connection management uses a two-tier Object Pool Pattern. HiveClientPool manages a fixed-size pool of IMetaStoreClient instances with connection lifecycle management (create, reconnect, close, connection exception detection). CachedClientPool adds a caching layer on top using Caffeine, keyed by user identity and configuration, enabling multi-tenant scenarios where different users get isolated client pools.

Two-Tier Connection Pool

```
CachedClientPool (ClientPool interface)
|
+-- Caffeine Cache<Key, HiveClientPool>
|
|   +-- Key = (metastoreURI, catalog, [UGI|userName], [conf:*])
|   |
|   +-- expireAfterAccess(evictionInterval)
|   |   removalListener -> pool.close()
|   |   scheduler -> background cleanup thread
|   |
|   +-- get(key) -> HiveClientPool (lazy creation)
|       |
|       +-- HiveClientPool extends ClientPoolImpl
|           |
|           +-- Pool size: configurable (default 5)
|           |
|           +-- newClient():
|               |   RetryingMetaStoreClient.getProxy(conf)
|               |
|           +-- reconnect(client):
|               |   client.reconnect()
|               |
|           +-- close(client):
|               |   client.close()
|               |
|           +-- isConnectionException(e):
|               |   TTransportException check
```

5.4 Adapter Pattern - Schema Conversion

HiveSchemaUtil and HiveSchemaConverter together implement the Adapter Pattern, converting between Iceberg's type system (Types.IntegerType, Types.StructType, etc.) and Hive's type system (TypeInfo, PrimitiveTypeInfo, etc.). This allows the two incompatible type systems to work together seamlessly. The conversion handles edge cases like Hive BYTE/SHORT -> Iceberg INTEGER promotion with an autoConvert flag.

Hive Type	Iceberg Type	Notes
BOOLEAN	BooleanType	Direct mapping
BYTE/SHORT	IntegerType	Promoted when autoConvert=true

INT	IntegerType	Direct mapping
LONG/BIGINT	LongType	Direct mapping
FLOAT	FloatType	Direct mapping
DOUBLE	DoubleType	Direct mapping
DECIMAL(p,s)	DecimalType(p,s)	Precision preserved
STRING/VARCHAR/CHAR	StringType	All mapped to string
BINARY	BinaryType	Direct mapping
DATE	DateType	Direct mapping
TIMESTAMP	TimestampType	Without timezone
LIST	ListType(E)	Recursive conversion
MAP	MapType(K,V)	Recursive conversion
STRUCT<...>	StructType	Recursive, field IDs assigned

Table 5.1: Hive-to-Iceberg Type Mapping

5.5 Decorator Pattern - CachedClientPool

CachedClientPool implements the same ClientPool interface as HiveClientPool, but adds a caching layer on top. It delegates all run() calls to the underlying HiveClientPool obtained from the Caffeine cache. This is a classic Decorator Pattern where the wrapped object and the wrapper share the same interface, allowing transparent enhancement.

5.6 Builder Pattern - ViewAware/TableAware Builders

HiveCatalog uses inner builder classes (ViewAwareTableBuilder and TableAwareViewBuilder) that extend the base builders to add cross-type conflict detection. ViewAwareTableBuilder checks if a view with the same name already exists before creating a table, and vice versa. This prevents accidental name collisions between tables and views in HMS, which share the same namespace.

6. Core Class Analysis

6.1 HiveCatalog

HiveCatalog is the primary entry point for all Iceberg operations backed by Hive Metastore. It extends BaseMetastoreViewCatalog (which itself extends BaseMetastoreCatalog) and implements SupportsNamespaces for database management and Configurable for Hadoop configuration injection.

Key Responsibilities:

- Table lifecycle: buildTable, listTables, dropTable, renameTable, tableExists, registerTable
- View lifecycle: buildView, listViews, dropView, renameView, viewExists
- Namespace lifecycle: createNamespace, listNamespaces, dropNamespace, loadNamespaceMetadata
- Namespace properties: setProperties, removeProperties (with owner/ownerType validation)
- Factory methods: newTableOps -> HiveTableOperations, newViewOps -> HiveViewOperations
- Location resolution: defaultWarehouseLocation (database location priority > warehouse dir)
- Cross-type conflict detection via ViewAwareTableBuilder and TableAwareViewBuilder

Design insight: The listTables method has two modes controlled by list-all-tables property. When false (default), it filters tables by checking the table_type=ICEBERG parameter, requiring batch HMS calls. When true, it returns all tables regardless of type, which is faster but less precise. The namespace model is flattened to one level (Hive databases), rejecting multi-level namespaces.

6.2 HiveTableOperations

HiveTableOperations is the most complex class in this module. It manages the full lifecycle of Iceberg table metadata in HMS. The class handles commit concurrency, encryption support, and metadata integrity verification.

Critical Implementation Details:

- doCommit: 400+ lines covering lock acquisition, conflict detection, HMS persistence, and error recovery with CommitStatus tracking (SUCCESS/FAILURE/UNKNOWN)
- doRefresh: Reads metadata_location from HMS, loads JSON from storage, handles encryption key integrity with SHA-256 hash verification
- Encryption: Supports table-level encryption with KEK/DEK architecture. Table key ID MUST come from HMS (trusted source), not from metadata file (untrusted storage)
- temp(): Returns anonymous TableOperations for uncommitted metadata, used during transaction building

- `lockObject()`: Strategy selection - `MetastoreLock` for lock-enabled configs, `NoLock` otherwise
- Metadata refresh max retries: configurable via `iceberg.hive.metadata-refresh-max-retries` (default 2)

6.3 MetastoreLock

`MetastoreLock` implements a sophisticated two-tier distributed lock mechanism. The JVM-level lock prevents unnecessary HMS lock contention from the same process, while the HMS-level lock provides cross-process/cross-node coordination.

Internal Components:

- `commitLockCache`: Static Caffeine Cache, evicted after 10 minutes of inactivity. Keyed by full table name. Ensures fair ordering with `ReentrantLock(true)`.
- `Heartbeat` (inner class): Implements `Runnable`, scheduled at fixed rate. Calls `client.heartbeat(0, lockId)` to keep the HMS lock alive. Records any exception for `ensureActive()` checks.
- `LockInfo` (inner class): Simple holder for `lockId` and `lockState`, used during lock acquisition flow.
- `WaitingForLockException` (inner class): Sentinel exception for retry loop control when lock state is `WAITING`.
- `agentInfo`: UUID-based identifier ('Iceberg-UUID') set on lock requests. Used by `findLock()` to recover locks after network failures (Hive 2+ only).

6.4 CachedClientPool

CachedClientPool provides a multi-tenant-aware caching layer over HiveClientPool. The cache key supports flexible composition: metastore URI + catalog name + optional UGI or username + arbitrary configuration values. This enables scenarios where different users or different catalog configurations should use separate connection pools.

The cache is a singleton (static field with synchronized init), using Caffeine with time-based eviction and a custom removal listener that calls `pool.close()` to release all `IMetaStoreClient` connections in the evicted pool. A scheduled executor thread ensures timely cleanup even when the cache is not actively accessed.

6.5 HMSTablePropertyHelper

HMSTablePropertyHelper is a stateless utility class that bridges Iceberg table metadata and HMS table properties. It serves as the single point of truth for what gets exposed to HMS and how. Key features include:

- Property translation: Maps `gc.enabled` -> `external.table.purge` for Hive compatibility
- Size-aware exposure: Respects `maxHiveTablePropertySize` (default 32672 chars). Properties exceeding the limit are silently skipped with a warning log
- Schema/PartitionSpec/SortOrder: Serialized to JSON and stored in HMS when within size limits
- Snapshot statistics: Exposes `current_snapshot_id`, `timestamp`, `summary`, `total files/records/size`
- Metadata hash: SHA-256 hash of the full TableMetadata JSON for encryption integrity verification
- Storage handler: Sets `HiveIcebergStorageHandler` when `hiveEngineEnabled` is true, enabling Hive query engine access

6.6 HiveOperationsBase

HiveOperationsBase is a package-private interface with default methods that provides shared functionality for both `HiveTableOperations` and `HiveViewOperations`. This avoids code duplication while respecting the different inheritance hierarchies of the two operation classes. Key default methods include `loadHmsTable` (load table from HMS via client pool), `persistTable` (create or alter table in HMS with optional CAS context), `newHmsTable` (create a properly configured HMS Table object), and static validation methods for type checking (`isValidIcebergTable`/`isValidIcebergView`). The `hmsEnvContext` method builds the environment context map used for lockless CAS operations in Hive 2+ where the expected `metadata_location` is passed as part of the `alter_table` call.

7. Key Scenario Call Chains

7.1 CREATE TABLE

CREATE TABLE Full Call Chain

```
// Entry: User calls catalog.buildTable(id, schema).create()

HiveCatalog.buildTable(identifier, schema)
  -> new ViewAwareTableBuilder(identifier, schema)

ViewAwareTableBuilder.create()
  -> viewExists(identifier) // cross-type conflict check
  -> super.create() // BaseMetastoreCatalog
  -> newTableOps(identifier)
  -> new HiveTableOperations(conf, clients, fileIO, kmc, name, db, tbl)
  -> TableMetadata.newTableMetadata(schema, spec, sortOrder, location, props)
  -> ops.commit(null, metadata)
  -> doCommit(null, metadata)
  -> writeNewMetadataIfRequired(true, metadata)
  -> io().newOutputFile(metadataFileLocation).createOrOverwrite()
  -> TableMetadataParser.write(metadata, output)
  -> lockObject(metadata) -> MetastoreLock | NoLock
  -> lock.lock()
  -> acquireJvmLock() -> commitLockCache.get(fullName)
  -> acquireLock() -> metaClients.run(client.lock(request))
  -> new Heartbeat().schedule(executor)
  -> loadHmsTable() -> null (table doesn't exist)
  -> newHmsTable(owner) -> new Table(name, db, owner, ...)
  -> storageDescriptor(schema, location, hiveEngineEnabled)
  -> HMSTablePropertyHelper.updateHmsTableForIcebergTable(
    newMetaLoc, tbl, metadata, removedProps, ...)
  -> lock.ensureActive()
  -> persistTable(tbl, false, envContext)
  -> metaClients.run(client.createTable(tbl))
  -> lock.ensureActive()
  -> [finally] cleanupMetadataAndUnlock(io, status, metaLoc, lock)
```

Figure 7.1: Complete Call Chain for CREATE TABLE

7.2 INSERT / APPEND DATA

INSERT / APPEND DATA Call Chain

```
// Entry: After writing data files, engine calls table.newAppend().commit()

table.newAppend()
  -> new MergingSnapshotProducer()
  -> .appendFile(dataFile).commit()
    -> produce new Snapshot with updated manifest list
    -> ops.commit(currentMetadata, updatedMetadata)

HiveTableOperations.doCommit(base=current, metadata=updated)
  -> encryptionPropsFromMetadata(metadata.properties())
  -> writeNewMetadataIfRequired(false, metadata)
    -> Writes v{N+1}.metadata.json to table/metadata/ directory
  -> lockObject(base) -> MetastoreLock
  -> lock.lock() // two-tier: JVM + HMS
  -> loadHmsTable() -> existing Table object
  -> VERIFY: base.metadataFileLocation == tbl.params['metadata_location']
    (if mismatch -> CommitFailedException: concurrent modification)
  -> HMSTablePropertyHelper.updateHmsTableForIcebergTable(
    newMetaLoc, tbl, metadata, removedProps, hiveEngineEnabled, ...)
  -> Set metadata_location = newMetaLoc
  -> Set previous_metadata_location = oldMetaLoc
  -> Update snapshot stats (id, timestamp, summary, files, records)
  -> Update schema, partition spec, sort order JSON
  -> lock.ensureActive()
  -> persistTable(tbl, true, envContext)
    -> MetastoreUtil.alterTable(client, db, name, tbl, envContext)
      -> Reflection-based call to HMS alter_table
      -> With DO_NOT_UPDATE_STATS = true
  -> lock.ensureActive()
  -> [finally] cleanupMetadataAndUnlock()
```

Figure 7.2: Complete Call Chain for Data Append

7.3 ALTER TABLE (Schema Evolution)

Schema Evolution Call Chain

```
// Entry: table.updateSchema().addColumn('new_col', Types.StringType()).commit()

table.updateSchema()
  -> new SchemaUpdate(ops)
  -> .addColumn('new_col', Types.StringType())
  -> .commit()
    -> Build updated Schema with new column (incremented field ID)
    -> Build updated TableMetadata with new schema
    -> ops.commit(current, updated)

HiveTableOperations.doCommit(base, metadata)
  -> ... (same commit flow as INSERT)
  -> Key difference in HMSTablePropertyHelper:
    -> storageDescriptor updated with new schema columns
      -> HiveSchemaUtil.convert(metadata.schema())
        -> For each Iceberg field -> Hive FieldSchema(name, type, comment)
    -> parameters['current-schema'] = SchemaParser.toJson(schema)
  -> persistTable(tbl, true, envContext)
    -> HMS table columns are updated
    -> metadata_location points to new metadata with evolved schema
```

Figure 7.3: ALTER TABLE (Schema Evolution) Call Chain

7.4 DROP TABLE

DROP TABLE Call Chain

```
// Entry: catalog.dropTable(identifier, purge=true)

HiveCatalog.dropTable(identifier, purge=true)
  -> isValidIdentifier(identifier) // must be 1-level namespace
  -> ops = newTableOps(identifier) -> HiveTableOperations
  -> lastMetadata = ops.current() // load for purge
    -> doRefresh()
      -> metaClients.run(client.getTable(db, name))
      -> refreshFromMetadataLocation(metadataLoc)
    -> return current TableMetadata
  -> clients.run(client.dropTable(db, name, false, false))
    // deleteData=false (Iceberg handles data deletion)
    // failOnNonExistent=false
  -> if purge && lastMetadata != null:
    CatalogUtil.dropTableData(ops.io(), lastMetadata)
    // Deletes: data files, manifest files, manifest lists,
    //          metadata files, statistics files
```

Figure 7.4: DROP TABLE Call Chain

7.5 Concurrent Write Conflict Resolution

Concurrent Write Conflict Resolution

```
// Scenario: Writer-B attempts commit while Writer-A just committed

Writer-B: HiveTableOperations.doCommit(base_v1, metadata_v3)
  -> writeNewMetadataIfRequired() -> writes v3.metadata.json OK
  -> lock.lock() -> acquired (Writer-A already released)
  -> loadHmsTable() -> tbl (with metadata_location = v2.metadata.json)
  -> VERIFY: base_v1.metadataFileLocation [v1.metadata.json]
    != tbl.params['metadata_location'] [v2.metadata.json]
  -> throw CommitFailedException(
    'Base metadata location v1 is not same as current v2')

// Error handling in the catch block:
  -> catch(Throwable e):
    -> If message contains 'The table has been modified...':
      -> checkCommitStatusStrict(newMetaLoc, metadata)
      -> Re-read table from HMS
      -> Compare metadata_location with newMetaLoc
      -> if FAILURE: throw CommitFailedException
    -> Otherwise:
      -> checkCommitStatus(newMetaLoc, metadata) // lenient
      -> if SUCCESS: silent return (was actually committed)
      -> if FAILURE: rethrow original exception
      -> if UNKNOWN: throw CommitStateUnknownException

// Finally block (always executed):
  -> cleanupMetadataAndUnlock(io, FAILURE, v3.metadata.json, lock)
  -> io.deleteFile(v3.metadata.json) // cleanup orphaned file
  -> lock.unlock()
```

Figure 7.5: Conflict Detection and Recovery Chain

8. Configuration Reference

Configuration Key	Default	Description
iceberg.hive.lock-timeout-ms	180000	Max time to wait for HMS lock acquisition
iceberg.hive.lock-check-min-wait-ms	50	Min wait between lock status checks
iceberg.hive.lock-check-max-wait-ms	5000	Max wait between lock status checks
iceberg.hive.lock-creation-timeout-ms	180000	Max time for lock creation with retries
iceberg.hive.lock-heartbeat-interval-ms	240000	HMS lock heartbeat interval (4 min)
iceberg.hive.table-level-lock-evict-ms	600000	JVM lock cache eviction timeout (10 min)
iceberg.hive.table-property-max-size	32672	Max HMS property value size (0=disable)
iceberg.hive.metadata-refresh-max-retries	2	Max retries for metadata refresh from storage
engine.hive.enabled	false	Enable Hive engine StorageHandler/SerDe
hive.lock-enabled	true	Enable HMS distributed locking
client.pool.size	5	HMS client pool size per cache entry
client.pool.cache.eviction-interval-ms	300000	Client pool cache eviction (5 min)
client.pool.cache-keys	(none)	Cache key elements: ugi, user_name, conf:*

Table 8.1: Complete Configuration Reference

9. Summary

The Apache Iceberg hive-metastore module is a well-designed integration layer that bridges the Iceberg table format with the Hive Metastore. Through careful analysis of 17 source files totaling approximately 3,500 lines of code, we can identify the following key architectural insights:

9.1 Architecture Principles

- **Metadata-Pointer Pattern:** HMS stores only the pointer (`metadata_location`) to the latest Iceberg metadata file. Actual metadata lives on distributed storage. This decouples the catalog service from the data format.
- **Optimistic Concurrency Control:** The commit process uses a write-then-verify pattern. Metadata is written to storage first, then the pointer is atomically updated in HMS under a distributed lock. Failed commits clean up orphaned metadata files.
- **Two-Tier Locking:** JVM-level `ReentrantLock` prevents intra-process contention; HMS-level `EXCLUSIVE` lock provides inter-process coordination. Heartbeat keeps the distributed lock alive.
- **Cross-Type Safety:** `ViewAwareTableBuilder` and `TableAwareViewBuilder` prevent accidental name collisions between tables and views, which share the same HMS namespace.
- **Encryption Integrity:** Table key IDs are read from HMS (trusted) rather than metadata files (untrusted). SHA-256 hash verification ensures metadata file integrity for encrypted tables.

9.2 Design Pattern Summary

Pattern	Where Applied	Purpose
Strategy	<code>HiveLock</code> / <code>MetastoreLock</code> / <code>NoLock</code>	Pluggable lock mechanism selection
Template Method	<code>BaseMetastoreTableOps</code> -> <code>HiveTableOps</code>	Standard commit/refresh algorithm with HMS-specific hooks
Object Pool	<code>HiveClientPool</code> + <code>CachedClientPool</code>	Efficient HMS connection reuse with caching
Adapter	<code>HiveSchemaUtil</code> / <code>HiveSchemaConverter</code>	Iceberg-Hive type system bridging
Decorator	<code>CachedClientPool</code> wrapping <code>HiveClientPool</code>	Add caching to connection pool transparently
Builder	<code>ViewAwareTableBuilder</code> / <code>TableAwareViewBuilder</code>	Cross-type conflict detection during creation

Table 9.1: Design Pattern Summary

This module demonstrates excellent separation of concerns, with each class having a clear single responsibility. The use of interfaces with default methods (`HiveOperationsBase`) elegantly solves the diamond inheritance problem between table and view operations. The connection pool hierarchy (`CachedClientPool` -> `HiveClientPool` -> `ClientPoolImpl`) provides production-ready

connection management with multi-tenant support. The lock mechanism is designed for resilience with heartbeat, retry, and recovery from network failures. Overall, this module serves as a reference implementation for how to integrate Iceberg with an external metadata catalog service.